Task 2 - Recursive Utility Functions

I denne oppgaven lager vi to rekursive funksjoner. Den første funksjonen finner loggfiler i mapper og undermapper. Den andre funksjonen undersøker JSON-data og finner aktive kasseregistre.

Rekursjon betyr at en funksjon kaller på seg selv for å løse en oppgave.

In [1]:
```
from pathlib import Path
import json
```

Task 2.1 - Recursive Directory Search

Her skal vi lage en funksjon som finner alle filer med filendelsen .log i en mappe og dens undermapper. Vi bruker Path fra pathlib og rekursjon for å undersøke mappestrukturen. Funksjonen skal returnere en lise med Path-objekter.

In [2]:
```
def find_log_files(folder: Path) -> list[Path]:
    """
    Finner alle log filene i en mappe og undermappene.
    Args: (folder: Path) : Mappen som skal undersøkes.
    Returns: list[Path]: en liste med filbaner tl loggfilene.
    """

    folder = Path(folder)
    log_files = []

# Går igjennom alle elementene i mappen
    for item in folder.iterdir():

        # Hvis elemtet er en mappe, undersøkes den rekursivt
        if item.is_dir():
            log_files.extend(find_log_files(item))

        # Hvis elemtet er en .log fil, legges den til i listen
        elif item.is_file() and item.suffix == ".log":
            log_files.append(item)

    return log_files
```

In [3]:
```
log_files = find_log_files(Path("data"))

print("Antall loggfiler:", len(log_files))

for file in log_files:
    print(file)
```

```
Antall loggfiler: 9
data\2026\library_cafe\POS_01\register_sales.log
data\2026\library_cafe\POS_02\register_sales.log
data\2026\library_cafe\POS_03\register_sales.log
data\2026\science_building\POS_01\register_sales.log
data\2026\science_building\POS_02\register_sales.log
data\2026\science_building\POS_03\register_sales.log
data\2026\student_union\POS_01\register_sales.log
data\2026\student_union\POS_02\register_sales.log
data\2026\student_union\POS_03\register_sales.log
```

Task 2.2 - Recursive JSON Traversal

Lager en funskjon som undersøker JSON-data og finner ID ene til alle aktive kasseregistre.

JSON-filen inneholder både dictionaries og lister på flere nivåer, derfor bruker vi rekursjon for å undersøke hele datastrukturen.

Resultatet skal returneres som et set, sånn at vi unngår dublikanter.

In [4]:
```python
# Leser JSON filen med informasjon om butikker og kasseregistre
with open("data/store_config.json", "r", encoding="utf-8") as file:
    config = json.load(file)
```

In [5]:
```python
def find_active_registers(data: object) -> set[str]:

    """
    Finner ID ene til aktive kassregistre i en datastruktur.
    Funksjonen undersøker dictionares og lister rekursivt.
    Args: (data: object): Datene som skal undersøkes.
    Returns: set[str]: Et sett med id`er til aktive kasseregistre.
    """

    active_registers = set()

    # Hvis data er en dictionary undersøkes innholdet
    if isinstance(data, dict):

        # Legger til ID`en dersom kasseregisteret er aktivt
        if data.get("active") is True and "id" in data:
            active_registers.add(data["id"])

        # Undersøker alle verdiene i dictionaryen rekursivt
        for value in data.values():
            active_registers.update(find_active_registers(value))

    # Hvis data er en liste undersøkes hhvert element
    elif isinstance(data, list):
        for item in data:
            active_registers.update(find_active_registers(item))

    return active_registers
```

In [6]:
```python
active_registers = find_active_registers(config)
```

```
print("Aktive kasseregistre:", sorted(active_registers))
print("Antall aktive kasseregistre:", len(active_registers))
```

```
Aktive kasseregistre: ['POS_01', 'POS_02', 'POS_03']
Antall aktive kasseregistre: 3
```

Results and Discussion

The recursive directory search identified 9 log files across the nested folder structure. The JSON traversal identified 3 active cash registers: `POS_01`, `POS_02`, and `POS_03`. Both functions successfully handled nested structures and returned the expected types of results. Using recursion made it possible to examine multiple levels without manually specifying each folder or JSON entry.